

Replication Plan

ScaWL: Scaling k -WL (Weisfeiler–Lehman) Algorithms in Memory and Performance on Shared and Distributed-Memory Systems

OSTI ID: 2587225

Auto-generated Replication Blueprint

April 8, 2026

Contents

1	Paper Overview	2
2	Detailed Workflow	2
2.1	Phase 1: Algorithm Implementation	2
2.2	Phase 2: Parallelization	2
2.3	Phase 3: Dataset Preparation	2
2.4	Phase 4: Experiments	3
2.5	Phase 5: Validation	3
3	Datasets	3
4	Models, Tools, and Software	3
4.1	Hardware Requirements	4

Paper Overview

This paper presents ScaWL, a scalable implementation of the k -dimensional Weisfeiler–Lehman (k -WL) graph isomorphism algorithm that runs efficiently on shared-memory and distributed-memory parallel systems. The k -WL algorithm iteratively refines k -tuple colorings to test graph isomorphism, but has high memory and computational cost ($O(n^k)$ tuples). ScaWL introduces memory-efficient data structures, communication-avoiding parallelism, and demonstrates strong/weak scaling on benchmark graph datasets.

Detailed Workflow

Phase 1: Algorithm Implementation

1.1 Implement the standard k -WL algorithm.

- Enumerate all k -tuples of vertices.
- Assign initial colors based on isomorphism type.
- Iteratively refine colors based on multiset of neighbor colors.
- Terminate when coloring stabilizes.

1.2 Implement memory-efficient tuple representation.

- Compressed storage of color histograms.
- Lazy tuple enumeration to reduce peak memory.

Phase 2: Parallelization

2.1 Shared-memory parallelism (OpenMP or threading).

- Parallelize tuple enumeration and color refinement.
- Thread-safe hash maps for color assignment.

2.2 Distributed-memory parallelism (MPI).

- Partition tuples across MPI ranks.
- Communication of boundary colors between ranks.
- Load balancing strategies.

Phase 3: Dataset Preparation

3.1 Obtain benchmark graphs.

- CFI (Cai–Fürer–Immerman) graphs for hard isomorphism instances.
- Random regular graphs.
- Real-world graphs from the TU Dortmund / benchmark collections.

3.2 Prepare graph input formats compatible with ScaWL.

Phase 4: Experiments

- 4.1 Correctness verification:** run k -WL on known isomorphic/non-isomorphic graph pairs.
- 4.2 Strong scaling:** fix problem size, increase core/node count; plot runtime vs. cores.
- 4.3 Weak scaling:** increase problem size proportional to core count.
- 4.4 Memory scaling:** measure peak memory vs. k and n .
- 4.5 Compare against serial/naive implementations.**

Phase 5: Validation

- 5.1** Reproduce scaling plots from the paper.
- 5.2** Reproduce speedup and efficiency tables.
- 5.3** Verify that ScaWL produces identical colorings to the serial reference.

Datasets

Dataset / Source		Type	Location / Notes
CFI graphs		Synthetic	Generated programmatically (standard construction)
Random regular graphs		Synthetic	Generated with NetworkX or custom code
TU Dortmund benchmark		Public	https://chrsmrrs.github.io/datasets/
ScaWL repository	test graphs	Bundled	From the paper’s code repository

Models, Tools, and Software

Tool / Library	Version	Purpose / URL
ScaWL	latest	Paper’s implementation GitHub (see paper for URL)
C/C++ compiler	GCC ≥ 10 or Clang ≥ 12	Compilation of ScaWL
MPI (OpenMPI / MPICH)	system	Distributed-memory parallelism
OpenMP	system	Shared-memory parallelism
CMake	≥ 3.18	Build system
Python / NetworkX	≥ 3.8	Graph generation and verification
Matplotlib	≥ 3.4	Scaling and performance plots

Tool / Library	Version	Purpose / URL
nauty / Traces	latest	Reference graph isomorphism solver for validation https://pallini.di.uniroma1.it/

Hardware Requirements

- HPC cluster or multi-node system for distributed-memory scaling experiments.
- ≥ 64 GB RAM per node for large k -WL instances.
- For $k = 3$ on graphs with $n > 100$: memory can exceed 100 GB.